

SPECIAL_BLOG — Full-Stack AI Mega Blog Platform

Project Technical Documentation & Verification Report for College Placement

Candidate / Developer: Karan Datta	Core Stack: MERN (React 19, Node.js, Express, MongoDB)
GitHub Repository: https://github.com/karan686-jpg/SPECIAL_BLOG.git	Integrations: Google Gemini 2.0 AI, ImageKit CDN, JWT
Project Category: Capstone Full-Stack Web Application	Verification Target: Technical Placement & Academic Review

1. Executive Summary & Objective

SPECIAL_BLOG is a full-stack, AI-integrated digital blogging and content management platform built using the **MERN Stack (MongoDB, Express, React 19, Node.js)**. The project provides a comprehensive content-publishing ecosystem featuring **Generative AI assistance (Google Gemini)** for automated blog post generation, **Quill.js WYSIWYG text editing**, **ImageKit cloud CDN** for automated WebP media compression, secure **JWT-based dual-role authentication (User/Admin)**, interactive community engagement (real-time likes and threaded comments), and a dedicated **Creator Analytics Engine**.

2. Technical Stack & Architecture

Layer / Area	Technologies Used	Key Role & Implementation Details
Frontend Core	React 19, Vite, React Router v7	Modern component architecture, client-side routing, Fast Refresh via Vite.
Styling & UX	Tailwind CSS v4, Motion, Lucide Icons	Responsive layout, interactive animations, clean visual hierarchy, toasts.
Rich Text Editor	Quill.js, Marked	WYSIWYG article creation with heading tags, quotes, bold/italic, and code blocks.
Backend Server	Node.js, Express.js v5, CORS, Dotenv	Modular REST API architecture, middleware authentication pipelines, error handling.
Database & ODM	MongoDB, Mongoose ODM v9	Relational schema design, population queries, views/likes indexing, cascade cleanup.
Generative AI	Google Gemini API (gemini-2.0-flash)	Prompt-to-HTML automated blog generation with rich formatting and fallback logic.
Cloud Media CDN	ImageKit SDK, Multer	Direct image upload buffer processing, auto WebP compression (1280px), CDN caching.
Authentication	JSON Web Tokens (JWT), Auth Guards	Role-based access control separating public readers, verified authors, and admins.

3. Core Modules & Key Capabilities

- **AI Blog Synthesis Assistant:** Integrated with Google Gemini 2.0 Flash model. Writers can input any title/prompt to synthesize clean, semantic HTML articles directly into the Quill editor.
- **Rich WYSIWYG Editor Suite:** Integrated Quill.js editor supporting multi-level headers, lists, code blocks, and formatted text for rich reader experience.
- **Cloud Media Processing Pipeline:** Cover images uploaded through Multer are streamed to ImageKit, converted to WebP format, and served via global CDN edges.
- **Dual Role-Based Access Control:** Secure JWT authentication separating Standard Users (article creation, likes, comments) and Administrators (publish/unpublish, comment approvals, all-blog control).
- **Interactive Engagement & Analytics:** Atomic view counting on article load, like toggle mechanism, threaded comments, and creator analytics dashboard (total views, likes, best-performing post).
- **Production Build & Responsive UI:** Fully responsive mobile/desktop design powered by Tailwind CSS v4 and React 19.

4. Database Schema Design (MongoDB / Mongoose)

Collection / Model	Key Fields & Types	Design Rationale & Relations
Blog	title, subtitle, category, description (HTML), image (CDN URL), views (Number), likes ([User Ref]), isPublished (Bool), author (User Ref), authorName	Stores formatted content with references to author & liking users. Indexed for feed sorting.
Comment	blog (Blog Ref), user (User Ref), name, content, isApproved (Bool), createdAt	Relational links to parent blog and user. Cleaned up via cascade delete on blog removal.
User / Admin	name, email (unique), password, profileImage, role (User/Admin)	Secure credential storage with JWT generation for authenticated session management.

5. Key REST API Endpoints

Endpoint	Method	Auth Level	Operation & Response
/api/blog/all	GET	Public	Fetches all published blogs sorted by date.
/api/blog/:blogId	GET	Public	Returns single blog and atomically increments view count.
/api/blog/add	POST	User / Admin	Multer file upload + ImageKit CDN + database write.
/api/blog/ai-generate	POST	Public	Generates rich HTML article via Google Gemini 2.0 Flash.
/api/blog/like	POST	User Auth	Toggles user ID inside blog likes array.
/api/blog/comments	POST	Public	Fetches approved comments populated with user profile.
/api/blog/analytics	GET	User Auth	Aggregates author's total views, likes, and top article.
/api/user/register	POST	Public	Registers user profile with secure credentials.
/api/user/login	POST	Public	Authenticates credentials and returns JWT bearer token.
/api/admin/toggle-publish	POST	Admin Auth	Moderates visibility of any published article.

6. Project Verification & Local Setup Instructions

College placement evaluators and technical interviewers can verify and run the project locally:

• Step 1: Clone Repository from GitHub

```
git clone https://github.com/karan686-jpg/SPECIAL_BLOG.git
cd SPECIAL_BLOG/blog-app
```

• Step 2: Start Backend Server

```
cd server && npm install
Configure .env with MONGODB_URI, JWT_SECRET, IMAGEKIT_KEYS, GEMINI_API_KEY
Run: npm run dev (Starts on Port 3000)
```

• Step 3: Launch Frontend Client

```
cd ../client && npm install
Run: npm run dev (Vite launches on http://localhost:5173)
```

Candidate Declaration: I hereby confirm that this project SPECIAL_BLOG is my authentic full-stack web application development work, implementing modern software engineering standards, MERN architecture, RESTful API design, and Generative AI integration.	Academic / Placement Evaluation: Evaluator Signature: _____ Evaluator Name / Designation: _____ Date of Verification: _____
---	---